14/09/2026

Experiment - 6 : Error detection & correction using Hamming Code

Aim: To implement error detection & correction using Hamming Code

Description :-

* Hamming code adds redundant (parity) bits to the data so the receiver can find & correct a single-bit error.
* Parity bits go at positions 1, 2, 4, 8 (powers of 2). Here 8 data bits + 4 parity bits = 12-bit codeword.
* Each parity bit uses even parity over the positions whose binary form contains that bit (P1 : 1, 3, 5, 7, 9, 11; P2 : 2, 3, 6, 7, 10, 11; P4 : 4, 5, 6, 7, 12; P8 : 8, 9, 10, 11, 12)
* At the receiver, the parity checks are recomputed. The value they form is the error position & 0 means no error.

Algorithm :-

* Sender : Convert text to 8-bit binary. For each character, place the data-bits in the non-power-of-2 positions, calculate P1, P2, P4, P8 and write the 12-bit codeword to "channel.txt".

* Receiver : Read "Channel.txt" & recompute the parity bits. If the result is non-zero, display the error position & flip that bit. Remove the parity bits & convert the binary back to ASCII.

Program Code :-

(i) sender.py

```
def encode_nibble_8bit(nibble):
    d1, d2, d3, d4 = nibble

    # Standard Hamming (7, 4) parity calculations
    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p4 = d2 ^ d3 ^ d4

    # 7-bit codeword
    c7 = [p1, p2, d1, p4, d2, d3, d4]

    # 8th-bit
    p0 = 0
    for b in c7:
        p0 ^= b

    # Complete 8-bit codeword
    c8 = c7 + [p0]
    return "".join(str(b) for b in c8)
```

```
text = input("Enter text: ")
channel_data = []
for char in text:
    binary_char = format(ord(char), '08b')
    nibble1 = [int(b) for b in binary_char[:4]]
    nibble2 = [int(b) for b in binary_char[4:]]

    channel_data.append(encode_nibble_8bit(nibble1))
    channel_data.append(encode_nibble_8bit(nibble2))

with open("channel", "w") as f:
    f.write(" ".join(channel_data))

print("Data successfully saved to channel as 8-bit blocks.")
```

(ii) receiver.py

```
def decode_nibble_8bit(code, block_idx):
    c = [int(b) for b in code]

    # Syndrome check using the first 7 bits
    s1 = c[0] ^ c[2] ^ c[4] ^ c[6]
    s2 = c[1] ^ c[2] ^ c[5] ^ c[6]
    s4 = c[3] ^ c[4] ^ c[5] ^ c[6]

    error_pos = s4*4 + s2*2 + s1*1

    if error_pos != 0:
        byte_pos = (block_idx // 2) + 1
        part = "Upper nibble" if (block_idx % 2 == 0)
                              else "Lower nibble"
        overall_bit_pos = (block_idx * 8) + error_pos

        print(f"[!] Error detected:")
        print(f"    - Block {block_idx + 1}")
        print(f"    - Bit position in block
                (1 to 8): {error_pos}")

        c[error_pos - 1] ^= 1   # Correct the bit

    data_bits = [c[2], c[4], c[5], c[6]]
    return "".join(str(b) for b in data_bits)
```

```
with open ("channel", "r") as f:
    blocks = f.read().split()

output_text = ""
for i in range (0, len(blocks), 2):
    high_nibble = decode_nibble_8bit (blocks[i], i)
    low_nibble = decode_nibble_8bit (blocks[i+1], i+1)
    byte_str = high_nibble + low_nibble
    output_text += chr (int (byte_str, 2))

print ("\n Decoded Output: ", output_text)
```

Input :-

Enter text : Hello !!

Data successfully saved to channel as 8-bit blocks.

Output :-

[!] Error detected :

- Block 2
- Bit position in block (1 to 8) : 1

Decoded Output : Hello !

Result : The Hamming code program detected the position of the single-bit error & corrected it, recovering the original message.

14/7/25